

PCD - Programação Concorrente e Distribuída

Sincronização de processos ligeiros

Luís Mota

September 30, 2024

Estrutura

- 1 Gestão de memória
- 2 Interferência
- 3 Exclusão mútua
- 4 Sincronização
- 5 Variáveis atómicas

Partilha de Memória I

Aplicação multi-thread

Memória

Thread 1

PC &
Stack

Thread 2

PC &
Stack

Thread 3

PC &
Stack

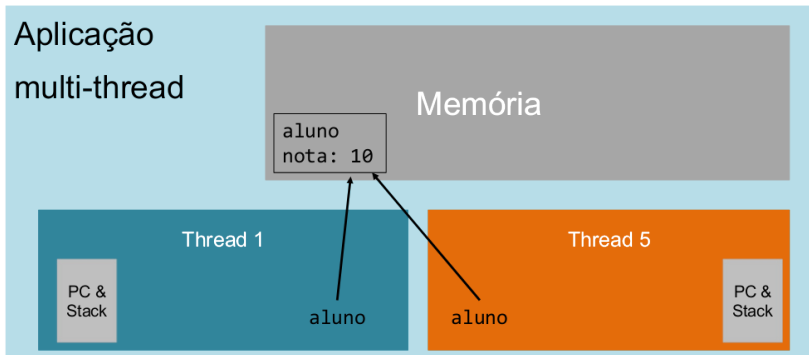
Thread 4

PC &
Stack

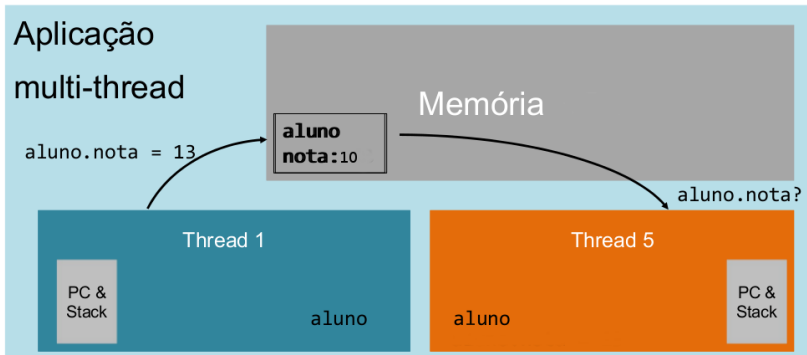
Thread 5

PC &
Stack

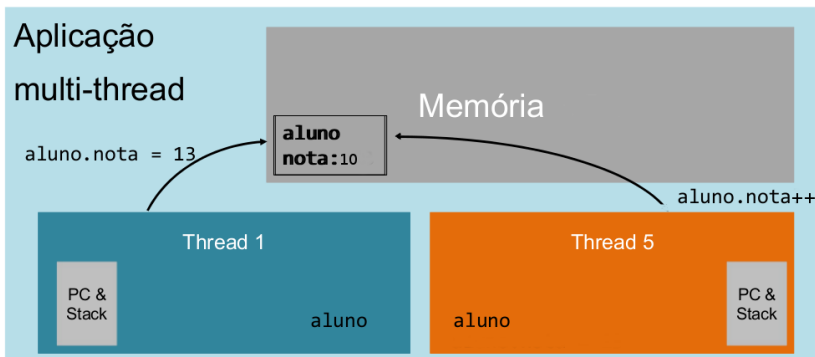
Partilha de Memória II



Partilha de Memória III



Partilha de Memória IV



Estrutura

- 1 Gestão de memória
- 2 Interferência**
- 3 Exclusão mútua
- 4 Sincronização
- 5 Variáveis atómicas

Partilha de referências entre *threads*

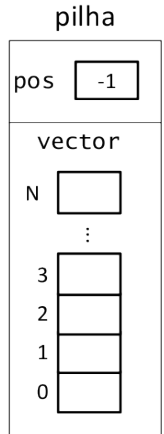
- Se estiverem a correr 2 ou mais *threads*, a ordem por que são executadas as diversas ações das *threads* pode variar de execução para execução.
- Podem ser gerados resultados diferentes de cada vez que é executado o programa, devido à ordem imprevisível por que são executadas as instruções

Race Condition

- Quando o valor de uma variável (ou o resultado do programa) variam devido à alteração da ordem de execução das ações das diferentes *threads*, temos um problema chamado “*race condition*”.
- Estas variabilidade e imprevisibilidade são normalmente nocivas e indesejáveis, e geram inconsistências.
- Em português diz-se por vezes “Condição de corrida”. Talvez fosse preferível “situação de concorrência” ou “situação de disputa”, mas é habitual usar o termo em inglês.

Pilha de Inteiros

```
class Pilha {  
    private int pos = -1;  
    private int vector[];  
  
    ...  
  
    public void push(int i) {  
        pos++;  
        vector[pos]=i;  
    }  
  
    public int pop() {  
        if (pos>=0){  
            return(vector[pos--]);  
        } else  
            throw new IllegalStateException() ;  
    }  
}
```



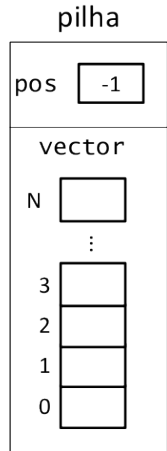
Pilha de Inteiros

Exemplo de execução paralela

Thread A
`pilha.push(5);`

Thread B
`pilha.push(7);`

```
public void push(int i) {  
    pos++;  
    vector[pos]=i;  
}
```



Pilha de Inteiros

Thread A
`pilha.push(5);`



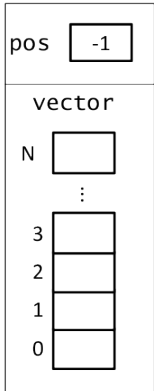
Thread B
`pilha.push(7);`



➡

```
public void push(int i) {  
    pos++;  
    vector[pos]=i;  
}
```

pilha



Pilha de Inteiros

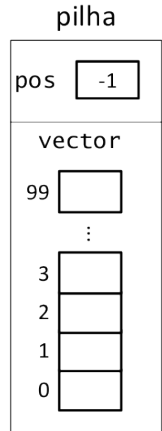
Thread A
`pilha.push(5);`

Thread B
`pilha.push(7);`



⇒

```
public void push(int i) {  
    pos++;  
    vector[pos]=i;  
}
```



Pilha de Inteiros

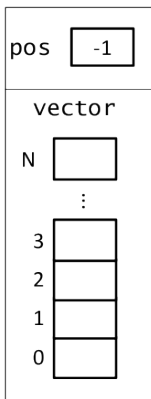
Thread A
`pilha.push(5);`

Thread B
`pilha.push(7);`



```
→ public void push(int i) {  
  → pos++;  
    vector[pos]=i;  
}
```

pilha



Pilha de Inteiros

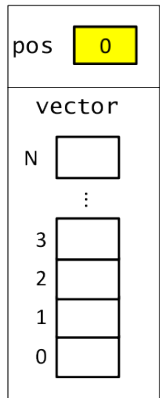
Thread A
`pilha.push(5);`

Thread B
`pilha.push(7);`



```
→ public void push(int i) {  
  → pos++;  
    vector[pos]=i;  
}
```

pilha



Pilha de Inteiros

Thread A
`pilha.push(5);`

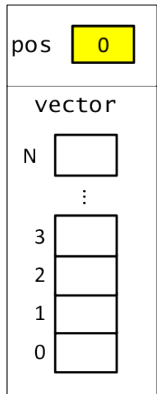


Thread B
`pilha.push(7);`



```
→ public void push(int i) {  
→     pos++;  
→     vector[pos]=i;  
→ }
```

pilha



Pilha de Inteiros

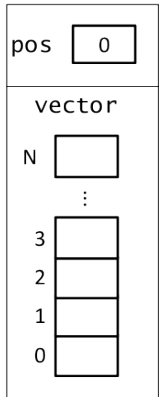
Thread A
`pilha.push(5);`



Thread B
`pilha.push(7);`

```
→ public void push(int i) {  
→     pos++;  
→     vector[pos]=i;  
→ }
```

pilha



Pilha de Inteiros

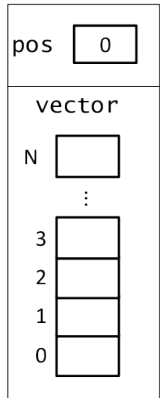
Thread A
`pilha.push(5);`



Thread B
`pilha.push(7);`

```
public void push(int i) {  
    → pos++;  
    → vector[pos]=i;  
}
```

pilha



Pilha de Inteiros

Thread A
`pilha.push(5);`



Thread B
`pilha.push(7);`

```
public void push(int i) {  
    → pos++;  
    → vector[pos]=i;  
}
```

pilha

pos

1

vector

N



⋮

3



2



1



0



Pilha de Inteiros

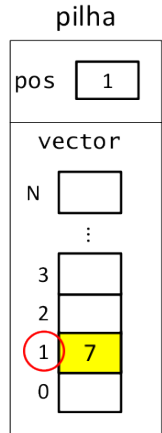
Thread A
`pilha.push(5);`



Thread B
`pilha.push(7);`

```
public void push(int i) {  
    pos++;  
    vector[pos]=i;  
}
```

Orange arrow points to the first line of the code block.
Green arrow points to the second line of the code block.



Pilha de Inteiros

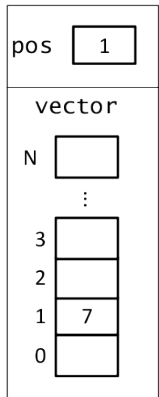
Thread A
`pilha.push(5);`



Thread B
`pilha.push(7);`

```
public void push(int i) {  
    pos++;  
    vector[pos]=i;  
}
```

pilha

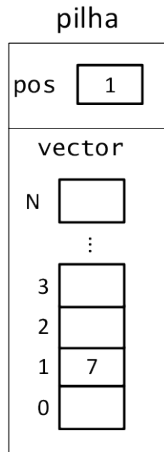


Pilha de Inteiros

Thread A
`pilha.push(5);`

Thread B
`pilha.push(7);`

```
public void push(int i) {  
    ➡ pos++;  
    vector[pos]=i;  
}
```

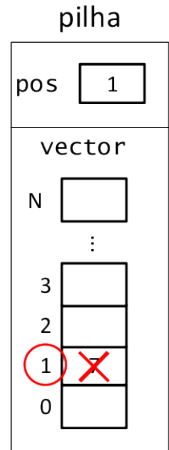


Pilha de Inteiros

Thread A
`pilha.push(5);`

Thread B
`pilha.push(7);`

```
public void push(int i) {  
    pos++;  
    ➡ vector[pos]=i;  
}
```

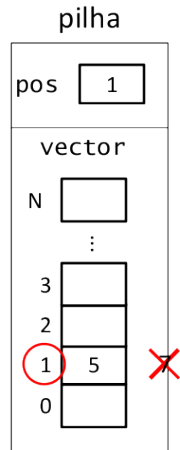


Pilha de Inteiros

Thread A
`pilha.push(5);`

Thread B
`pilha.push(7);`

```
public void push(int i) {  
    pos++;  
    ➡ vector[pos]=i;  
}
```

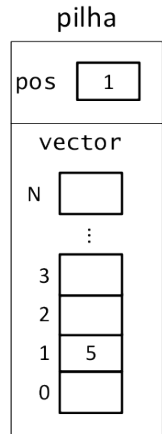


Pilha de Inteiros

Thread A
`pilha.push(5);`

Thread B
`pilha.push(7);`

```
public void push(int i) {  
    pos++;  
    vector[pos]=i;  
    → }  
}
```

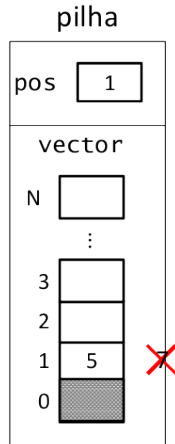


Pilha de Inteiros

Thread A
`pilha.push(5);`

Thread B
`pilha.push(7);`

```
public void push(int i) {  
    pos++;  
    vector[pos]=i;  
}
```



Outra vista sobre a mesma interferência

Thread A	Thread B	pos	vector
		-1	{0,0,...}
pos++		0	{0,0,...}
	pos++	1	{0,0,...}
	vector[pos]=7	1	{0,7,...}
vector[pos]=5		1	{0,5,...}

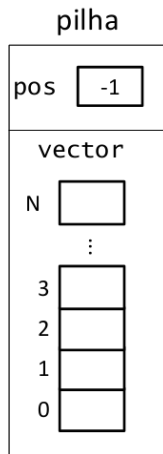
Pilha de Inteiros: push e pop

Exemplo de execução paralela

Thread A
`pilha.push(5);`

Thread B
`pilha.pop();`

```
public void push(int i) {  
    1 pos++;  
    4 vector[pos]=i;  
}  
  
public int pop(){  
    2 if(pos>=0)  
    3 pos --;  
}
```



Outra vista sobre o problema push-pop

Thread A	Thread B	pos	vector
		-1	{0,0,...}
pos++		0	{0,0,...}
	if(pos ≥ 0)	0	{0,0,...}
	pos - -	-1	{0,0,...}
vector[pos]=5		-1	ERRO

Ações Atómicas: definição

- Ações não atómicas, ou compostas, podem ser divididas em ações atómicas.
- Ações atómicas correm num único ciclo do processador e por isso não podem ser interrompidas.

Ações Atómicas: exemplos intuitivos

- $x+1$

- $x=4$

Não atómicas:

- $x=x+5$

- $x++$

- invocação de métodos

- $x=x+1$

Interferência entre *threads*

- Diz-se que pode existir interferência entre duas *threads* quando as suas ações de alteração sobre determinados dados decorrem através de sequências de ações atómicas.
- Também pode haver interferência entre ações de alteração e leitura, sobretudo quando a leitura é complexa e envolve várias ações.
- Em sequências de ações atómicas, uma *thread* pode ser intercalada ou executada em paralelo com outra *thread* e, eventualmente, aceder aos dados num estado intermédio ou incompleto.

Inconsistência

Acontece quando duas threads estão a trabalhar com valores diferentes, inválidos ou inconsistentes para os mesmos dados.

Secção Crítica

Secção crítica é uma sequência de ações cuja execução, se interrompida ou intercalada, corre o risco de criar inconsistências.

```
pos++;
```

```
pilha[pos]=i;
```

Estrutura

- 1 Gestão de memória
- 2 Interferência
- 3 Exclusão mútua**
- 4 Sincronização
- 5 Variáveis atómicas

Exclusão Mútua

Cadeado ou Mutex (*mutual exclusion*) é o mecanismo base da programação concorrente para evitar inconsistências. Limita o acesso a uma sequência de código, permitindo apenas acesso exclusivo de um único processo ligeiro.



```
public void push(int i) {  
    pos++;  
    pilha[pos]=i;  
}
```

Cadeado *Mutex*

- Tem dois estados:

– Livre



– Ocupado



- Tem duas operações, trancar e destrancar:

– Lock

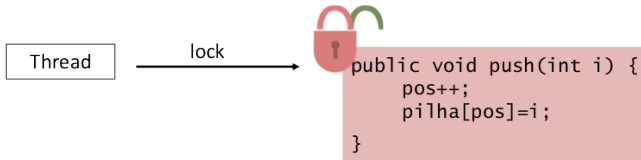


– Unlock



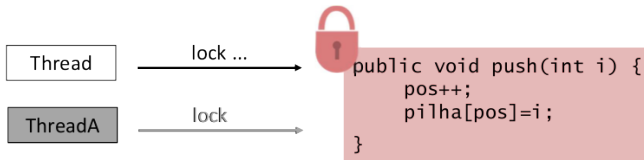
Trancar

O cadeado só pode ser trancado por um processo ligeiro de cada vez.



Trancar II

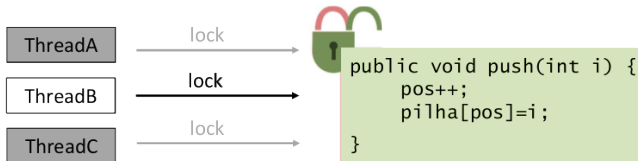
Se um processo ligeiro tentar trancar um cadeado que já está trancado (por outro processo ligeiro), a operação ficará em espera até que este último seja destrancado.



Trancar III

Se vários processos ligeiros estiverem à espera de um cadeado trancado, apenas um poderá prosseguir quando este for destrancado.

O processo ligeiro que terá acesso ao cadeado é escolhido arbitrariamente.



Estrutura

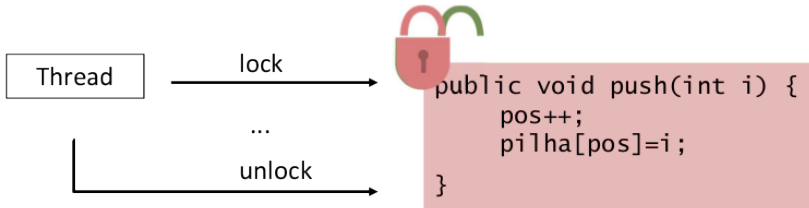
- 1 Gestão de memória
- 2 Interferência
- 3 Exclusão mútua
- 4 Sincronização**
- 5 Variáveis atómicas

Sincronização

- É a aplicação de um cadeado a troços de código, normalmente em secções críticas
- O processo ligeiro deve trancar o cadeado que protege a secção crítica antes de a executar
- Quando termina esta secção, o cadeado é libertado

Com o mecanismo de sincronização, passa a evitar-se que vários processos ligeiros tenham acesso simultâneo (ou intercalado) à secção crítica.

Sincronização



Sincronização em Java

- Em Java, todos os objetos têm associado um cadeado *mutex* intrínseco. Em inglês, é comum ser designado *monitor*.
- Para definir uma secção crítica, usa-se a palavra reservada `synchronized`
- A sincronização processa-se no âmbito da instância, não do método ou classe.
- Em Java pode sincronizar-se:
 - métodos
 - blocos de código.
- É da responsabilidade do programador definir as secções críticas, e protegê-las através da sincronização.

Métodos Sincronizados

Nos métodos sincronizados, a sincronização é feita com o cadeado intrínseco do objeto.

```
public synchronized void push(int i) {  
    pos++;  
    pilha[pos]=i;  
}
```

The diagram illustrates the lock mechanism for the `push` method. A horizontal line connects the `synchronized` keyword to a box labeled `Lock (this)`. Another horizontal line connects the closing curly brace `}` to a box labeled `UnLock (this)`.

Métodos sincronizados

```
class Pilha {  
    private int pos = -1;  
    private int pilha[]= new int[100];  
    public synchronized void push(int i) {  
        pos++;  
        pilha[pos]=i;  
    }  
    public synchronized int pop() {  
        if (pos>=0){  
            return(pilha[pos--]);  
        } else  
            throw new IllegalStateException() ;  
    }  
}
```

Blocos Sincronizados

Nos blocos sincronizados, é necessário especificar o objeto cujo cadeado intrínseco será usado para sincronizar.

```
private LinkedList<String> employees= ...  
public void addEmployee(String name){  
    synchronized(employees) {  
        ←  
        employees.add(name);  
        ←  
    }  
    ...  
}
```

Lock (employees)

UnLock (employees)

Blocos Sincronizados

Nos blocos sincronizados, pode também escolher-se a instância implícita para fazer a sincronização. (É aliás a situação mais normal.)

```
public void addEmployee(String name) {  
    synchronized(this){  
        employees.add(name);  
    }  
}
```


De novo a pilha

Definir o método push usando blocos sincronizados:

```
public void push(int i) {  
    synchronized(this){  
        pos++;  
        pilha[pos]=i;  
    }  
}
```

Cadeado

Além dos cadeados intrínsecos, existem classes que implementam o interface Lock e permitem usar esta funcionalidade de forma mais explícita e flexível.

Método	Descrição
<code>void lock()</code>	Adquire o cadeado, senão bloqueia à espera.
<code>boolean tryLock()</code>	Tenta a aquisição dentro de um prazo (opcional), senão devolve falso.
<code>void unlock()</code>	Destranca o cadeado.

Pilha com cadeado

```
public class Pilha{  
    private int pos = -1;  
    private int pilha[]= new int[100];  
    private Lock lock=new ReentrantLock();  
    public void push(int i) {  
        lock.lock();  
        try{  
            pos++;  
            pilha[pos]=i;  
        } finally{  
            lock.unlock();  
        }  
    }  
    (...)  
}
```

Estrutura

- 1 Gestão de memória
- 2 Interferência
- 3 Exclusão mútua
- 4 Sincronização
- 5 Variáveis atómicas

Variáveis atómicas

- Existe um conjunto de classes que disponibilizam operações atómicas, ie, que não podem ser interrompidas;
- Exemplos: `AtomicInteger`, `AtomicIntegerArray`, `AtomicReference<V>`,...
- Encontram-se no pacote `java.util.concurrent.atomic`
- Podem, em casos limitados, ser alternativa à sincronização: se toda uma secção crítica puder ser substituída por uma ação atómica, não pode ocorrer interferência.

AtomicInteger

Alguns métodos disponíveis:

- `int addAndGet(int delta)`
- `boolean compareAndSet(int expect, int update)`
- `int incrementAndGet()`
- `int getAndDecrement()`
- ...

Mais leituras

Variáveis atómicas: docs.oracle.com/javase/tutorial/essential/concurrency/atomicvars.html

- 1 Gestão de memória
- 2 Interferência
- 3 Exclusão mútua
- 4 Sincronização
- 5 Variáveis atómicas